Why Dijkstra Uses a Priority Queue

Nov 4, 2026 · @mihir

Short answer

A plain FIFO queue with re-relaxation and stale-entry skipping gives correct shortest paths on positive weights. It is the label-correcting family (Bellman-Ford queue variant, often called SPFA). The priority queue does not add correctness; it adds a guarantee that makes the algorithm efficient.

What the priority queue guarantees

When you always pop the minimum-distance node, its distance is final the moment it is popped. It will never improve.

This holds because every weight is non-negative. All other nodes in the queue have distance at least as large as the popped node. Any path to the popped node that goes through them can only be longer or equal, since adding edges never reduces a total.

Consequences:

- Each node is finalized exactly once.
- A node's neighbours are never re-explored.
- Total work is O((V + E) log V).

What the FIFO version loses

With a plain queue you pop nodes in BFS order (fewest edges first), not in distance order. A popped node's distance is therefore only tentative: a path with more edges but a smaller total weight can still improve it later. That is why you need the re-check, the re-push and the stale-entry skip.

Example graph, source S:

Edge	Weight
S → A	10
S → B	1

Edge	Weight
B → C	1
C → A	1

BFS order reaches A directly, so A is popped early with distance 10 and its neighbours are explored using that value. Later the path S → B → C → A gives A a distance of 3. A is pushed again, and everything downstream of A must be re-explored with the better value. If A had a long chain behind it, that whole chain is reprocessed.

The cost

Your version is correct, but a node can be re-pushed many times, and each re-push can cascade through its descendants. So the gap to a heap is not just a log factor.

- Priority queue Dijkstra: O((V + E) log V) in every case.
- Plain FIFO label-correcting: O(V · E) worst case, like Bellman-Ford.
- Looser orderings (for example stack/DFS order) have constructed graphs where running time becomes exponential.

The conceptual reason

Dijkstra is a greedy algorithm, and the priority queue is what makes the greedy choice valid. The invariant is that the closest unfinalized node already sits at its true shortest distance. Extracting the minimum is the proof of correctness that lets you skip re-checking, not an optimization added on top.

Your version replaces that guarantee with a fix-it-later mechanism: re-relax and skip stale entries until nothing improves. This is the same idea as Bellman-Ford. It works in any order, and even with negative weights as long as there is no negative cycle, but you pay for not having a good order.

Summary

Both versions are correct on positive weights; the priority queue guarantees each node is processed once.

	Priority queue	Plain FIFO queue
Correct with positive weights	Yes	Yes
Node final when popped	Yes, once	No, can improve later
Re-exploration of nodes	Never	Possible, repeatedly
Worst case	$O((V + E) \log V)$	$O(V \cdot E)$ or worse